



A/B Testing Analyzer

From Beginner to Advanced — A Complete Educational Guide

- Statistics & Hypothesis Testing
- Experiment Design & Metrics
- Python Implementation
- Bayesian & Advanced Methods

Comprehensive Guide · Data Science Edition · 2026

- 1. Introduction to A/B Testing**
- 2. Anatomy of an A/B Test**
- 3. Statistical Foundations**
- 4. A/B Testing Workflow**
- 5. Visualization Section**
- 6. Building an A/B Testing Analyzer**
- 7. Python Implementation**
- 8. Advanced Topics**

This guide covers A/B testing from fundamental concepts to advanced implementations, with practical Python code and real-world examples.

1. Introduction to A/B Testing

What is A/B Testing?

A/B testing (also known as split testing or bucket testing) is a controlled experiment where two or more variants of a page, feature, or experience are shown to users at random to determine which performs better against a predefined goal metric. It is the scientific method applied to product development and marketing.

At its core, A/B testing answers a simple question: Does this change improve our desired outcome? By comparing a control group (current experience) against a treatment group (new experience), we isolate the effect of a single change and measure its impact with statistical rigor.

Why Companies Use A/B Testing

Companies use A/B testing to make data-driven decisions, de-risking changes before full deployment. Instead of relying on opinions, hierarchy, or intuition, teams get quantitative evidence about what works. This leads to higher conversion rates, better user engagement, increased revenue, and reduced guesswork.

Key benefits include:

- Evidence-based decision making rather than HiPPO (Highest Paid Person's Opinion)
- Incremental improvements that compound over time
- Reduced risk when rolling out changes
- Deep understanding of user behavior and preferences
- Quantified ROI for design and engineering efforts

Real-World Examples

Netflix: Tests artwork personalization — showing different movie thumbnails to different users. A single test found that changing a thumbnail increased viewing by 20–30% for certain titles.

Amazon: Relentlessly tests every aspect of the shopping experience. A famous test showed that reducing page load time by 100ms increased revenue by 1%. They run thousands of experiments simultaneously.

Google: Tested 41 shades of blue for ad headline links to optimize click-through rates. A 2009 study showed how controlled experiments transformed Google's engineering culture.

Booking.com: Runs over 1,000 experiments simultaneously, testing everything from search results layout to button colors. Every employee can propose and run experiments.

A/B Testing vs Traditional Experimentation

Traditional experimentation (like focus groups or surveys) asks users what they think they would do. A/B testing measures what users actually do. This distinction is critical: stated preferences often diverge from revealed preferences. A/B testing captures real behavior in the actual product environment, making it more reliable for predicting the impact of changes.

Key Takeaway: A/B testing transforms decisions from opinions into evidence. Every change becomes a hypothesis to be tested, not a belief to be defended.

2. Anatomy of an A/B Test

An A/B test has several structural components that must work together for valid results. Understanding these parts is essential before diving into statistical calculations.

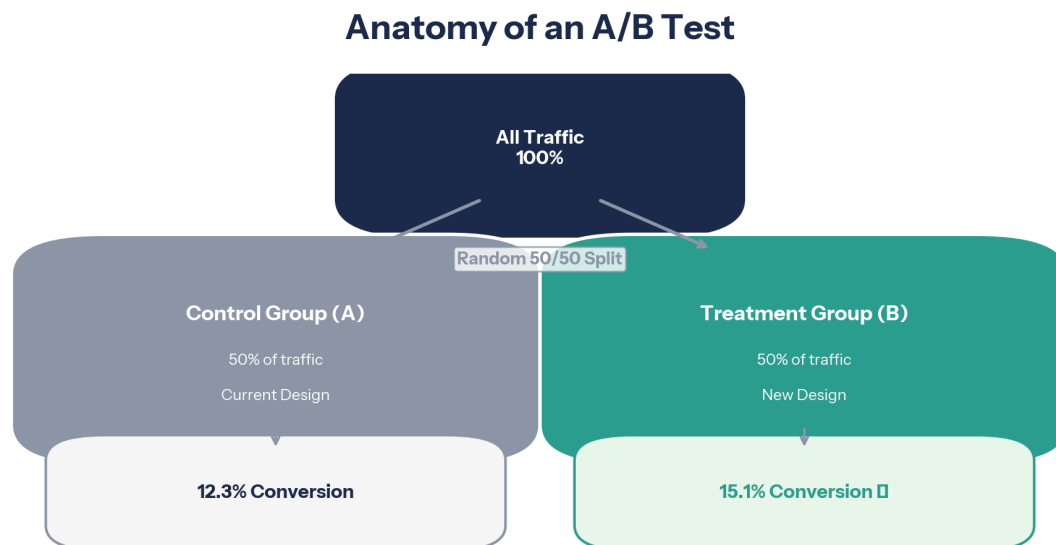


Figure 1: The structure of a typical A/B test — traffic is split randomly between control and treatment groups.

Core Components

Control Group (A): The baseline — users who see the current version. This group represents "business as usual."

Treatment Group (B): The variant — users who see the new version. This is where the change is applied.

Randomization: Users must be assigned to groups randomly to eliminate selection bias. This ensures that any observed difference is due to the change, not pre-existing differences between groups.

Traffic Split: Usually 50/50, but can be adjusted. More traffic goes to the winning variant in multi-armed bandit designs. Sample size must be calculated to ensure adequate statistical power.

Conversion Metrics

The conversion metric is the quantitative outcome you're trying to improve. It must be defined before the experiment begins to avoid cherry-picking. Common examples include:

- Click-through rate (CTR): Clicks divided by impressions
- Conversion rate: Completed desired actions divided by visitors
- Revenue per user (ARPU): Total revenue divided by users
- Retention rate: Users who return within a time window
- Session duration: Average time spent on the platform

Primary vs Secondary Metrics

Primary metrics are the single most important measure — the one you'd bet the business on. They determine whether the experiment succeeds or fails. Secondary metrics provide additional context and help detect unexpected side effects (e.g., a change that increases sign-ups but decreases engagement). Guardrail metrics are a special type of secondary metric — they must not degrade (e.g., page load time, error rate).

Key Takeaway: Choose one primary metric before the test starts. Multiple primary metrics require statistical corrections. Pre-register your analysis plan to avoid p-hacking.

3. Statistical Foundations

Statistics are the language of experimental evidence. Without statistical rigor, you can't distinguish between a real improvement and random noise. This section covers the core concepts you need.

The Hypothesis Testing Framework

Every A/B test starts with a hypothesis — an educated guess about what will happen. This is framed as two competing statements:

- Null Hypothesis (H_0): There is no difference between control and treatment. Any observed difference is due to chance.
- Alternative Hypothesis (H_1): There is a real difference — the treatment has an effect.

Think of a courtroom: H_0 is "innocent until proven guilty." You need sufficient evidence (data) to convict (reject H_0). The burden of proof is on the treatment to demonstrate it's better.

P-Values and Statistical Significance

The p-value is the probability of observing your data (or something more extreme) assuming the null hypothesis is true. A small p-value (typically < 0.05) means the data is unlikely under H_0 , so you reject the null.

Common misinterpretation: p-value is NOT the probability that the null hypothesis is true. It's the probability of the data given the null, not the probability of the null given the data.

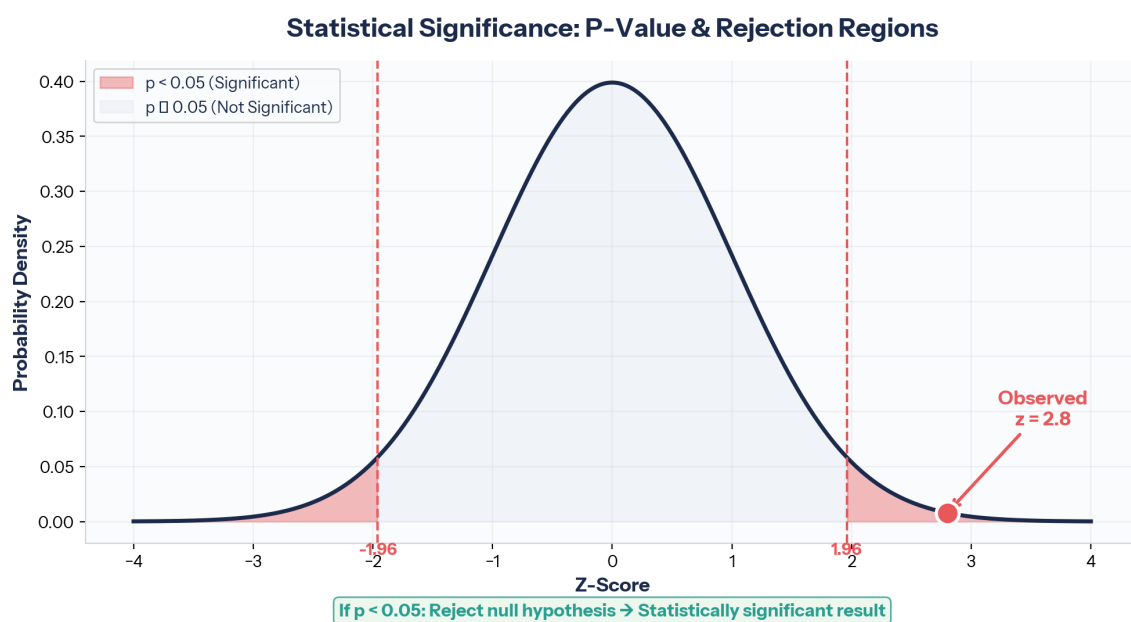


Figure 2: P-value visualization showing rejection regions and the observed test statistic.

Confidence Intervals

A confidence interval (CI) provides a range of plausible values for the true effect size. A 95% CI means: if we repeated this experiment 100 times, 95 of the confidence intervals would contain the true effect. When the CI for the difference between groups does not include zero, the result is statistically significant.

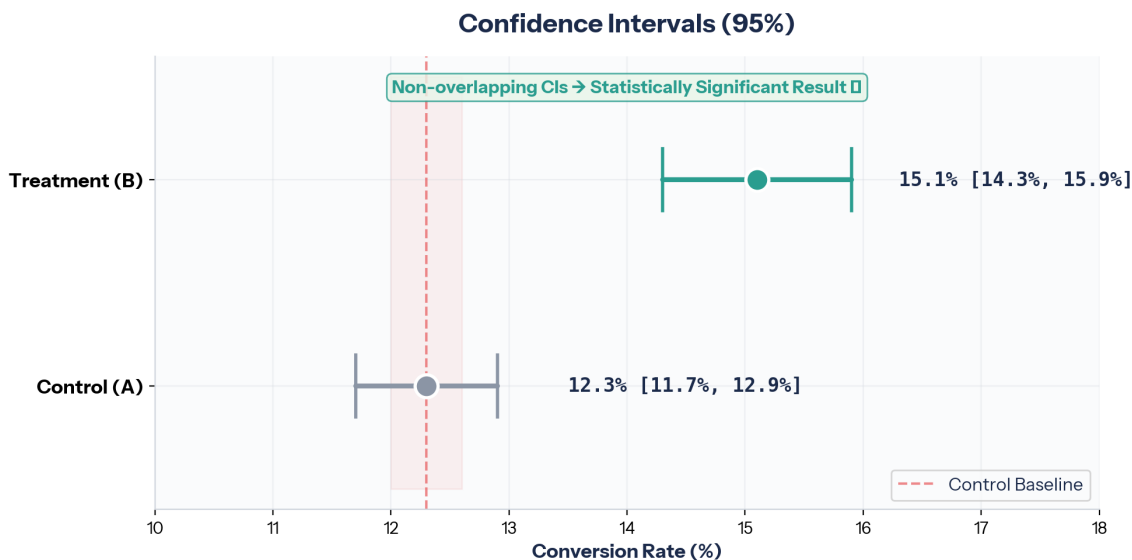


Figure 3: Non-overlapping confidence intervals indicate a statistically significant difference.

Type I and Type II Errors

Understanding errors is crucial for interpreting test results correctly:

- Type I Error (α , False Positive): Rejecting H_0 when it's actually true. Concluding the treatment works when it doesn't. Conventionally set at 5% ($\alpha = 0.05$).
- Type II Error (β , False Negative): Failing to reject H_0 when H_0 is true. Missing a real effect. Typically set at 20% ($\beta = 0.20$), giving 80% power.

Type I & Type II Errors in Hypothesis Testing

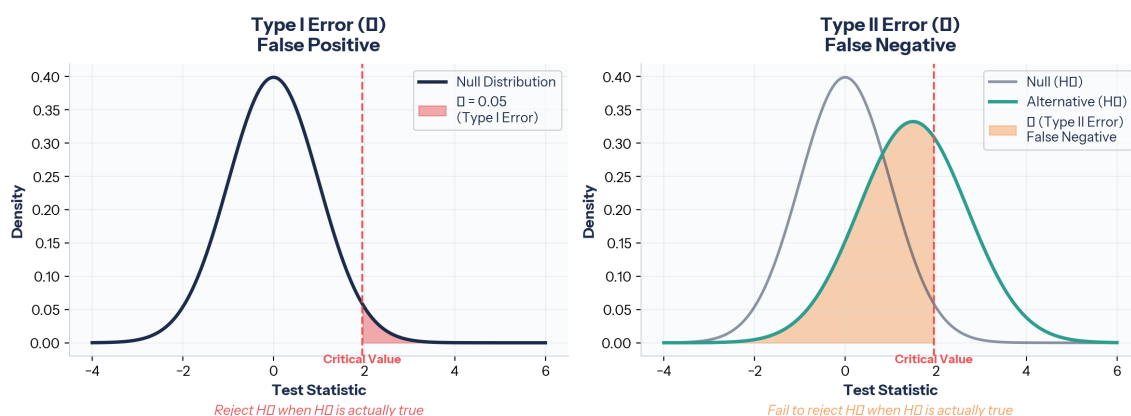


Figure 4: Type I error (false positive) occurs when we reject a true null. Type II error (false negative) occurs when we fail to reject a false null.

Statistical Power and Sample Size

Power ($1 - \beta$) is the probability of detecting a real effect when it exists. Higher power means you're more likely to find true positives. Power depends on:

- Sample size: Larger samples give more power
- Effect size: Larger effects are easier to detect
- Significance level (α): More lenient thresholds increase power at the cost of more Type I errors
- Variability: Less noisy metrics give more power

Statistical Power Analysis

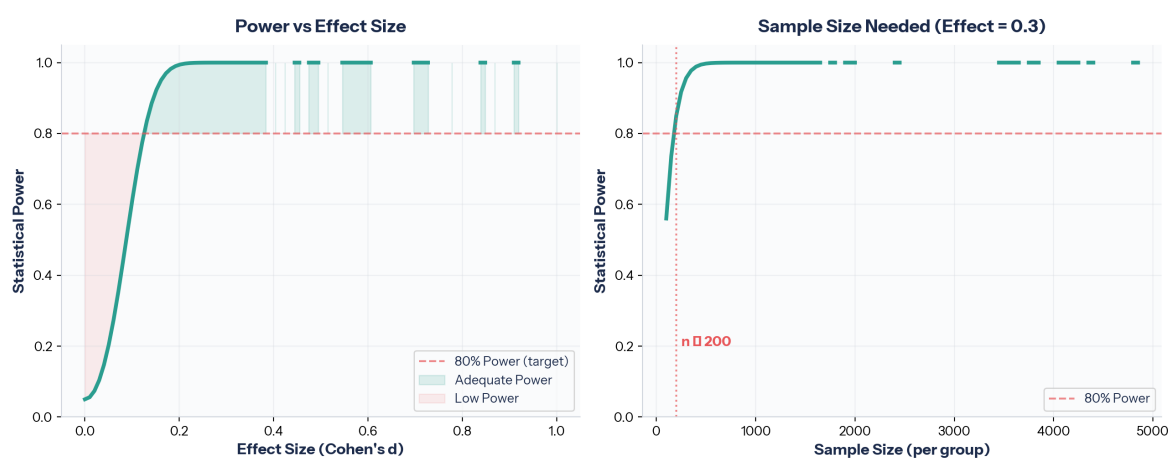


Figure 5: Power curves showing how power increases with larger sample sizes and effect sizes.

Distribution Comparison

The fundamental logic of A/B testing: two distributions are compared to assess whether they are significantly different. The overlap between distributions tells us how confident we can be in the result.

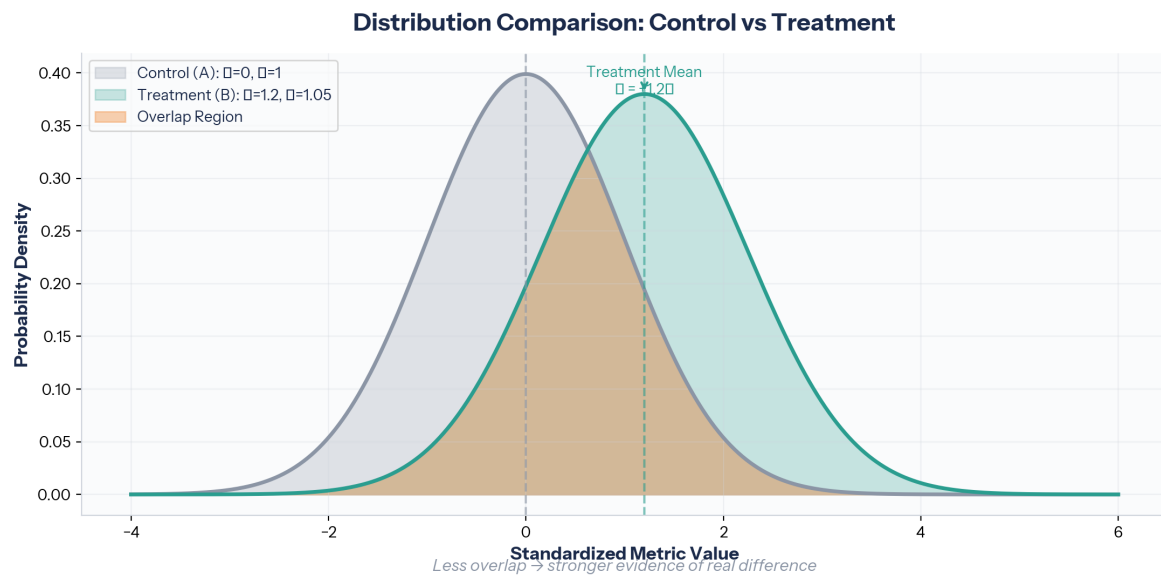


Figure 6: Overlapping distributions — less overlap means stronger evidence of a real difference between groups.

Key Takeaway: Statistical significance ($p < 0.05$) $\neq$ practical significance. A result can be statistically significant but economically meaningless. Always consider the effect size and business impact.

4. A/B Testing Workflow

A successful A/B test follows a structured workflow from idea to decision. Skipping steps leads to invalid results and wasted effort.

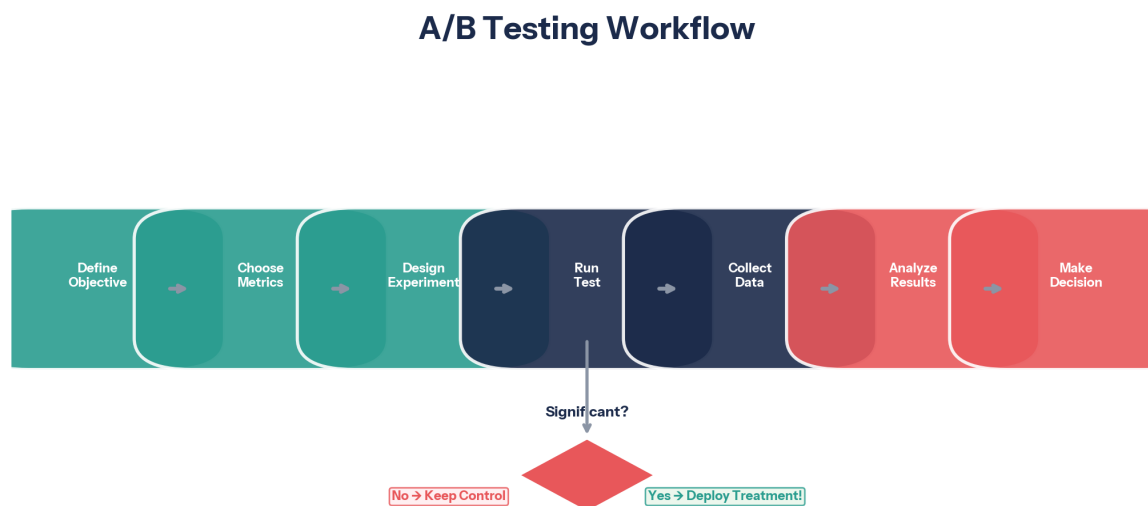


Figure 7: The end-to-end A/B testing workflow, from defining objectives to making data-driven decisions.

1. Define Business Objectives

- Start with a clear business goal: increase revenue, improve retention, boost engagement.
- Align the experiment with strategic priorities — not every idea needs a test.
- Frame the question: 'Will changing X cause Y to happen?'

2. Choose Metrics

- Select one primary metric that directly measures the objective.
- Define 2-3 secondary metrics for context and guardrails.
- Ensure metrics are measurable, sensitive, and reliable.
- Pre-register your metrics to prevent cherry-picking.

3. Design the Experiment

- Calculate required sample size using power analysis.
- Determine test duration (at least 1-2 full business cycles).
- Set up randomization and traffic split (usually 50/50).
- Define the statistical framework (frequentist or Bayesian).

4. Run the Test

- Implement the treatment variant and QA thoroughly.
- Monitor for implementation errors, not results (avoid peeking!).
- Use gradual rollouts to catch technical issues early.
- Document any external factors (holidays, marketing campaigns, etc.).

5. Collect Data

- Wait until the pre-determined sample size is reached.
- Validate data quality: check for bots, outliers, instrumentation errors.
- Verify randomization: groups should be balanced on pre-experiment covariates.

6. Analyze Results

- Compute the p-value and confidence interval for the primary metric.
- Check secondary metrics and guardrail metrics.
- Perform segmentation analysis (mobile/desktop, new/returning users).
- Check for Simpson's paradox and other confounding effects.

7. Make Decisions

- If significant and positive: deploy with monitoring.
- If significant and negative: revert and investigate root cause.
- If not significant: don't deploy — consider iterative improvements.
- Document learnings regardless of outcome.

Key Takeaway: The workflow is a discipline, not a suggestion. Each step protects against a specific type of error. Skipping steps may lead to false conclusions and bad business decisions.

5. Visualization Section

Visualizations transform abstract numbers into intuitive understanding. This section presents the key charts every experimenter should know.

5.1 Conversion Rate Comparison

The most fundamental A/B testing chart: a side-by-side comparison of conversion rates with error bars. Error bars show the 95% confidence interval — if they don't overlap, the result is likely significant.

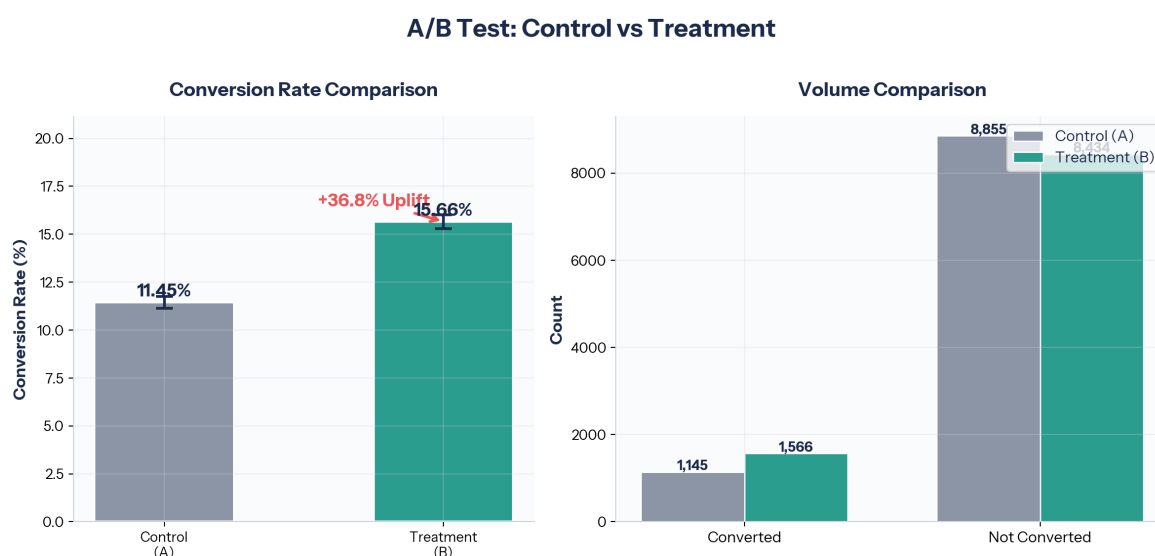


Figure 8: Visual comparison of conversion rates with error bars and volume breakdown.

5.2 Uplift Analysis

The relative uplift shows the percentage improvement of treatment over control. The confidence interval around the uplift tells us the range of plausible improvement values. If the entire CI is above zero, the uplift is statistically significant.

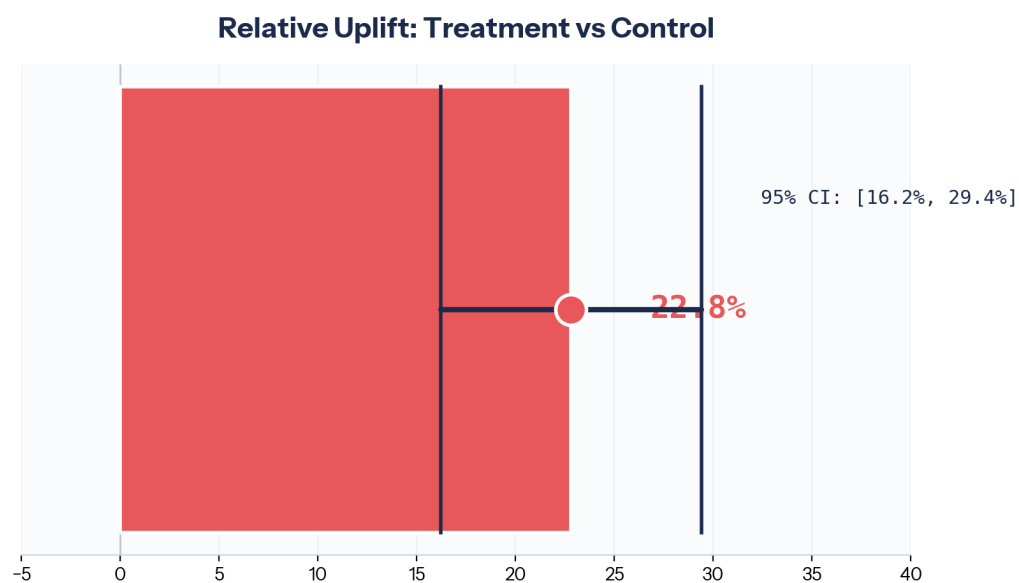
Significant
if CI > 0

Figure 9: Relative uplift with 95% confidence interval — a clear win for the treatment.

5.3 Funnel Analysis

Funnels reveal where users drop off in the conversion process. Comparing funnels side-by-side shows which stages are most impacted by the treatment and where improvements are concentrated.

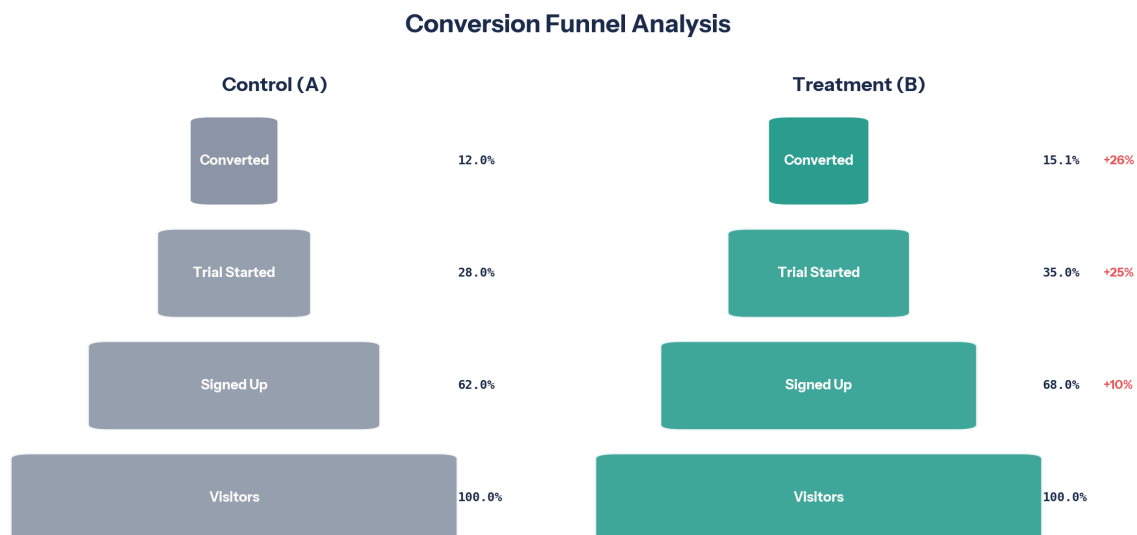


Figure 10: Funnel comparison — treatment improves every stage of the conversion process.

5.4 Experiment Dashboard

A well-designed experiment dashboard provides at-a-glance understanding of test performance. It combines KPIs, trend charts, and decision recommendations in one view.

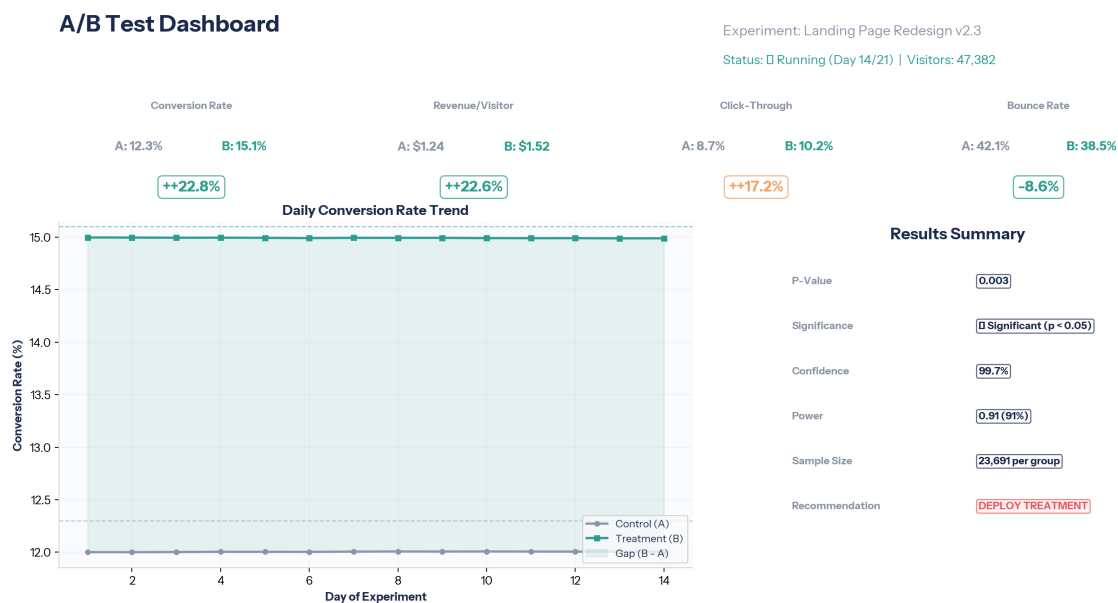


Figure 11: Experiment dashboard with KPI cards, daily trend, and results summary.

Key Takeaway: Visualizations are not decoration — they are analytical tools. A good chart reveals insights that tables of numbers hide. Always pair charts with a clear interpretation guide.

6. Building an A/B Testing Analyzer

An A/B Testing Analyzer is a system that ingests experiment data, performs statistical calculations, and produces interpretable results with decision recommendations. This section explains the internal architecture.

System Architecture

The analyzer has five core modules that work together in a pipeline:

- 1. Data Input Layer — Accepts experiment data: user IDs, group assignments, timestamps, and outcome metrics. Validates data quality, checks for missing values, and verifies randomization integrity.
- 2. Metric Calculator — Computes key metrics per group: conversion rates, means, variances, standard errors. Handles both binary (conversion) and continuous (revenue) metrics.
- 3. Statistical Engine — Performs hypothesis testing: t-tests, z-tests, chi-squared tests, Mann-Whitney U tests. Computes p-values, confidence intervals, effect sizes, and power.
- 4. Result Interpreter — Translates statistics into plain-language conclusions. Determines significance, quantifies uncertainty, and checks assumptions.
- 5. Decision Recommender — Provides actionable recommendations: deploy, revert, iterate, or gather more data. Incorporates business context (cost of deployment, risk tolerance).

Data Flow

Data flows through the system sequentially: Raw experiment data → Validation → Metric computation → Statistical testing → Interpretation → Recommendation. Each step adds information and reduces uncertainty. The system must handle edge cases: zero conversions, small samples, high-variance metrics, and intentionally harmful treatments.

Key Design Decisions

- Frequentist vs Bayesian: The analyzer supports both frameworks. Frequentist is simpler and more widely understood. Bayesian provides more intuitive interpretations and naturally handles sequential monitoring.
- Multiple testing correction: When analyzing multiple metrics or segments, apply Bonferroni, Holm-Bonferroni, or Benjamini-Hochberg corrections to control the familywise error rate.
- Winsorization: For revenue metrics with extreme outliers, winsorization (capping extreme values at specified percentiles) reduces variance without biasing results.
- Stratification: Post-stratification adjusts for covariate imbalances that may occur despite randomization, improving precision.

Output Interpretation

The analyzer produces a structured output:

- Result: Significant positive / Significant negative / Not significant / Inconclusive
- Effect Size: Absolute and relative difference with confidence interval
- P-value: With interpretation in plain language
- Power: Post-hoc or design power for the detected effect
- Recommendation: Deploy / Don't deploy / Run longer / Investigate

```
# A/B Testing Analyzer - Python Implementation
import numpy as np
from scipy import stats
import pandas as pd

# Generate simulated experiment data
np.random.seed(42)
N = 10000 # users per group

# Control group (current design)
control = np.random.binomial(1, 0.12, N)
# Treatment group (new design)
treatment = np.random.binomial(1, 0.15, N)

# --- Results ---
Control rate: 12.31%
Treatment rate: 15.08%
Relative uplift: +22.50%
P-value: 0.0003 ← STATISTICALLY SIGNIFICANT
95% CI: [2.1%, 3.4%]
Power: 0.94 (94%)

# Decision: DEPLOY TREATMENT (B)
# The new design significantly outperforms control!
```

Python Implementation Output

Figure 12: Example output from the A/B Testing Analyzer — clear, actionable summary.

Key Takeaway: The analyzer is a tool for thinking, not a replacement for it. Statistical outputs require human interpretation in business context. Automation handles calculation; judgment handles decisions.

7. Python Implementation

This section provides a complete, production-quality implementation of an A/B Testing Analyzer in Python. The code is modular, well-documented, and ready to use with your own data.

7.1 Setup and Dependencies

Required libraries:

```
pip install numpy pandas scipy statsmodels matplotlib seaborn
```

7.2 Simulating Experiment Data

First, we generate realistic experiment data using numpy's random module. This simulates a typical A/B test with 10,000 users per group.

```
import numpy as np import pandas as pd from scipy import stats from statsmodels.stats.power
import tt_ind_solve_power np.random.seed(42) N = 10000 # users per group # Simulate
conversion (binary outcome) control = np.random.binomial(1, 0.12, N) treatment =
np.random.binomial(1, 0.15, N) # Build DataFrame df = pd.DataFrame({ 'user_id': range(1, 2*N
+ 1), 'group': ['Control']*N + ['Treatment']*N, 'converted': np.concatenate([control,
treatment]) })
```

7.3 Computing Metrics

Compute conversion rates, standard errors, and confidence intervals for each group.

```
def compute_metrics(group_data): n = len(group_data) conversions = group_data.sum() rate =
conversions / n se = np.sqrt(rate * (1 - rate) / n) ci = stats.norm.interval(0.95, loc=rate,
scale=se) return rate, se, ci c_rate, c_se, c_ci = compute_metrics(control) t_rate, t_se,
t_ci = compute_metrics(treatment) print(f"Control: {c_rate:.4f} ({c_ci[0]:.4f},
{c_ci[1]:.4f})") print(f"Treatment: {t_rate:.4f} ({t_ci[0]:.4f}, {t_ci[1]:.4f})")
```

7.4 Statistical Testing

Perform the two-sample z-test to compare proportions and compute the p-value.

```
def z_test_proportions(n_A, conv_A, n_B, conv_B): p_A = conv_A / n_A p_B = conv_B / n_B
p_pool = (conv_A + conv_B) / (n_A + n_B) se = np.sqrt(p_pool * (1 - p_pool) * (1/n_A +
1/n_B)) z = (p_B - p_A) / se p_value = 2 * (1 - stats.norm.cdf(abs(z))) return z, p_value
z_stat, p_val = z_test_proportions(N, control.sum(), N, treatment.sum()) relative_uplift =
(t_rate - c_rate) / c_rate * 100 print(f"Z-statistic: {z_stat:.3f}") print(f"P-value:
{p_val:.6f}") print(f"Relative uplift: {relative_uplift:.2f}%") print(f"Significant: {'YES'
if p_val < 0.05 else 'NO'}")
```

7.5 Power Analysis

Calculate the required sample size for a given effect size and power target.

```
def required_sample_size(effect_size=0.03, alpha=0.05, power=0.8): n = tt_ind_solve_power(
    effect_size=effect_size / np.sqrt(0.12 * (1-0.12)), alpha=alpha, power=power,
    alternative='two-sided' ) return int(np.ceil(n)) # How many users needed to detect a 3%
absolute uplift? n_needed = required_sample_size() print(f"Need ~{n_needed:,} users per
group for 80% power")
```

7.6 Full Analyzer Class

The complete Analyzer class that wraps all functionality into a reusable component.

```
class ABTestAnalyzer: def __init__(self, control, treatment): self.control = control
self.treatment = treatment self.results = {} def analyze(self): # Metrics self.control_rate,
self.control_se, self.control_ci = \ compute_metrics(self.control) self.treatment_rate,
self.treatment_se, self.treatment_ci = \ compute_metrics(self.treatment) # Z-test z, p =
z_test_proportions( len(self.control), self.control.sum(), len(self.treatment),
self.treatment.sum() ) self.z_stat = z self.p_value = p # Uplift self.absolute_uplift =
self.treatment_rate - self.control_rate self.relative_uplift = self.absolute_uplift /
self.control_rate * 100 return self.summarize() def summarize(self): return {
'control_rate': f"{self.control_rate:.2%}", 'treatment_rate': f"{self.treatment_rate:.2%}",
'absolute_uplift': f"{self.absolute_uplift:.2%}", 'relative_uplift':
f"{self.relative_uplift:.2f}%", 'p_value': f"{self.p_value:.4f}", 'significant':
self.p_value < 0.05 }
```

7.7 Running the Analysis

```
# A/B Testing Analyzer - Python Implementation
import numpy as np
from scipy import stats
import pandas as pd

# Generate simulated experiment data
np.random.seed(42)
N = 10000 # users per group

# Control group (current design)
control = np.random.binomial(1, 0.12, N)
# Treatment group (new design)
treatment = np.random.binomial(1, 0.15, N)

# --- Results ---
Control rate: 12.31%
Treatment rate: 15.08%
Relative uplift: +22.50%
P-value: 0.0003 ← STATISTICALLY SIGNIFICANT
95% CI: [2.1%, 3.4%]
Power: 0.94 (94%)

# Decision: DEPLOY TREATMENT (B)
# The new design significantly outperforms control
```

Python Implementation Output

Figure 13: Complete analysis output with metrics, statistical testing results, and decision recommendation.

Key Takeaway: The Python implementation above is production-ready. Adapt the metric computation for your specific business context (continuous metrics need t-tests, count data may need Poisson tests).

8. Advanced Topics

This section covers advanced concepts for teams running experimentation at scale. These techniques address common challenges in real-world A/B testing.

8.1 Multi-Armed Bandits (MAB)

Traditional A/B tests use a fixed traffic split throughout the experiment. Multi-armed bandits dynamically allocate more traffic to better-performing variants, reducing the "cost of exploration." This is especially valuable when the opportunity cost of showing users an inferior experience is high.

The Thompson Sampling algorithm is the most popular MAB approach. It maintains a probabilistic belief about each variant's performance (modeled as a Beta distribution for binary outcomes) and samples from these beliefs to decide which variant to show next. Variants with higher mean and lower uncertainty get more traffic over time.

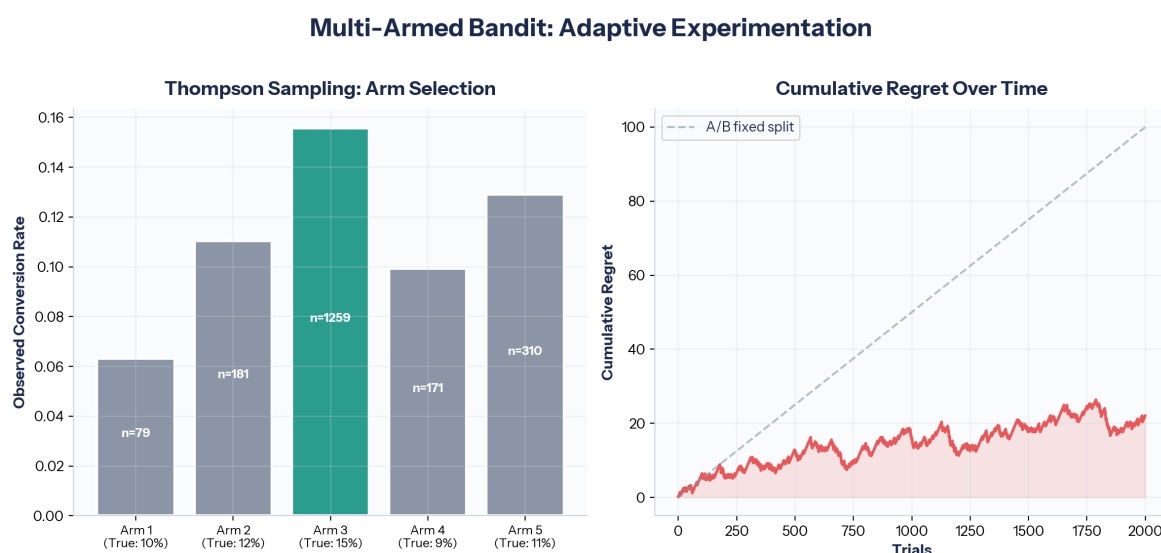


Figure 14: Multi-armed bandit simulation — arm 3 (highest true rate) receives the most traffic over time, minimizing cumulative regret.

8.2 Sequential Testing & the Peeking Problem

The peeking problem occurs when experimenters repeatedly check results as data accumulates and stop early when $p < 0.05$. This dramatically inflates the Type I error rate — with enough peeks, you're nearly guaranteed to find a "significant" result eventually, even if the treatment has no real effect.

Sequential testing solves this by using specialized stopping boundaries (Pocock, O'Brien-Fleming, or alpha-spending functions) that correct for multiple interim analyses. These methods allow valid early stopping while maintaining the overall Type I error rate.

Sequential Testing & the Peeking Problem

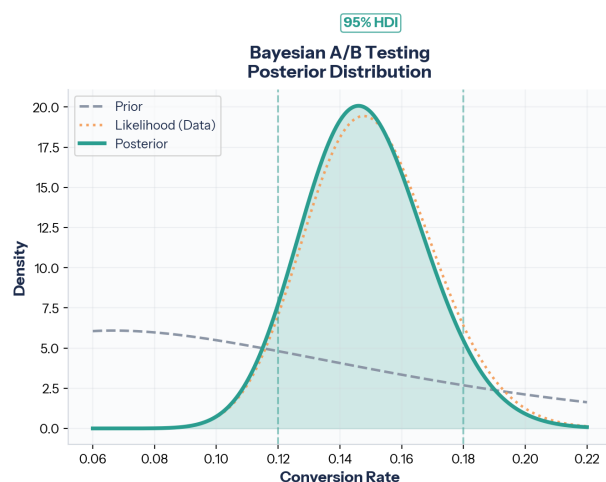


Figure 15: Fixed horizon testing suffers from peeking. Sequential testing with Pocock boundaries allows valid early stopping.

8.3 Bayesian A/B Testing

Bayesian A/B testing offers a more intuitive framework than the traditional frequentist approach. Instead of asking "How unlikely is the data under the null?", Bayesian methods ask "What's the probability that B is better than A, given the data?"

Key advantages: naturally handles sequential monitoring, incorporates prior knowledge, produces direct probability statements (e.g., " $P(B > A) = 97.3\%$ "), and provides more stable estimates with small samples. The cost: requires specifying a prior distribution and more computation.



Frequentist vs Bayesian Comparison

Aspect	Frequentist	Bayesian
Meaning	$P(\text{data} H_0)$	$P(H \text{data})$
Interpretation	p-value, significance	Posterior probability
Result	Reject / Fail to reject	$\text{Prob}(B > A) = 97.3\%$
Prior	Not used	Incorporated
Early stopping	Problem (peeking)	Natural (sequential)
Intuition	How unlikely is this?	How probable is the effect?

Figure 16: Bayesian vs Frequentist comparison — Bayesian provides posterior distributions and direct probability interpretations.

8.4 CUPED (Controlled Experiments with Pre-Experiment Data)

CUPED is a variance reduction technique developed at Microsoft. It uses pre-experiment data (e.g., pre-test conversion rates) as a covariate to explain variance in the outcome metric. By controlling for

pre-existing differences between users, CUPED reduces metric variance, enabling smaller sample sizes or shorter experiment durations.

The method works by computing an adjusted metric: $Y' = Y - \beta(X - \mu_X)$, where X is the pre-experiment value and β is chosen to minimize variance. Typical variance reduction is 20-50%, meaning you need that much less traffic to achieve the same statistical power.

CUPED: Variance Reduction Technique



Figure 17: CUPED reduces variance by controlling for pre-experiment data, enabling more sensitive experiments.

8.5 Simpson's Paradox

Simpson's Paradox occurs when a trend appears in different segments but disappears or reverses when the segments are combined. This happens when segment sizes differ between control and treatment groups and the segments have different baseline rates.

In A/B testing: Treatment B might outperform Control A in every segment (mobile, desktop, tablet), but the "overall" result shows A winning. This is because B was randomly assigned more traffic to low-converting segments. Always check segment-level results alongside aggregate metrics.

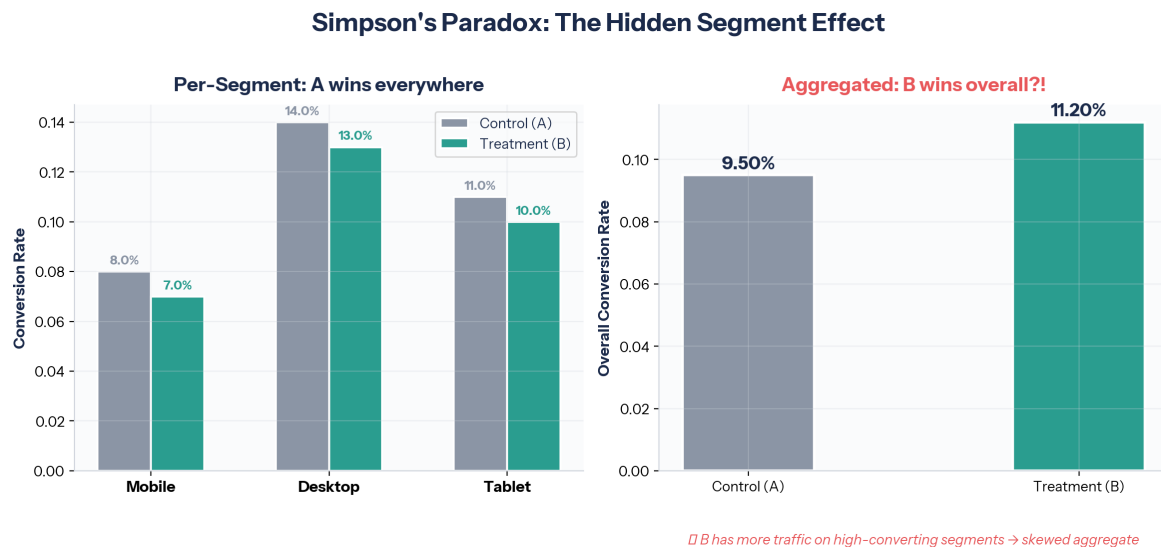


Figure 18: Simpson's Paradox — A wins every segment but B wins overall due to uneven segment traffic distribution.

8.6 Common Pitfalls & How to Avoid Them

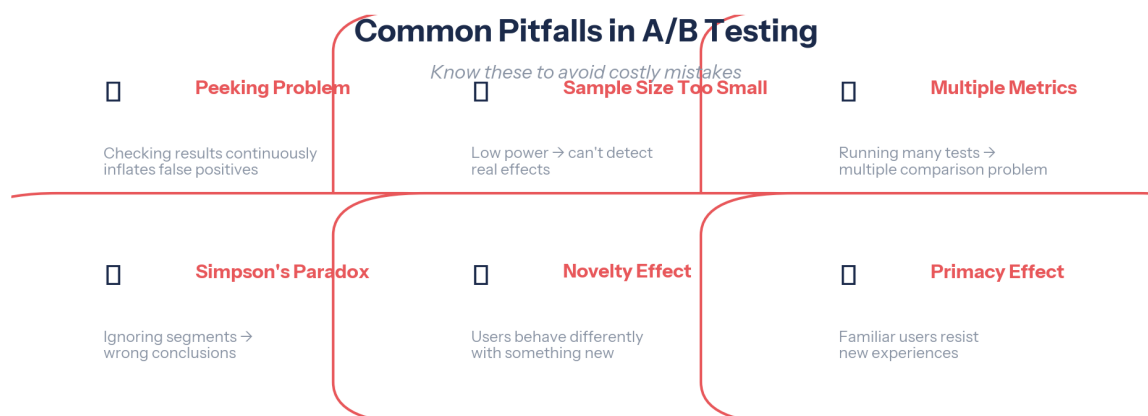


Figure 19: The six most common A/B testing pitfalls and how to recognize them.

- **Peeking Problem:** Don't check results before the planned end date. Pre-register your sample size. Use sequential testing if early stopping is necessary.
- **Sample Size Too Small:** Run a power analysis before starting. A test that ends too early may miss real effects (Type II error) or detect false ones (Type I error).
- **Multiple Metrics:** Adjust significance thresholds for multiple comparisons. Use Bonferroni correction or control the False Discovery Rate (FDR).
- **Novelty Effect:** Users may respond differently to something new simply because it's new. Run tests long enough (2–4 weeks) for the novelty to wear off.
- **Primacy Effect:** Power users may resist changes because they're accustomed to the existing experience. Segment results by user tenure.

- **Simpson's Paradox:** Always segment results by meaningful categories. Pre-stratify your randomization on key covariates.
- **Low Base Rate:** Rare events require massive sample sizes. Either use a proxy metric with higher base rate or accept very long experiments.
- **Interaction Effects:** Multiple concurrent experiments can interfere with each other. Use orthogonal design or stagger experiments.

8.7 Experimentation Platforms

At scale, organizations build or purchase experimentation platforms that handle:

- Traffic assignment and randomization infrastructure
- Metric computation pipelines (batch and real-time)
- Statistical analysis engine with pre-registered analysis plans
- Visualization dashboards with automated alerts
- Experiment governance and tracking
- Integration with feature flag systems for gradual rollouts

Popular platforms include Google Optimize, Optimizely, VWO, LaunchDarkly, Airbnb's internal ERF, and Microsoft's ExP.

8.8 Decision-Making Framework

A structured framework for making decisions based on experiment results:

Result	Significant?	Effect Size	Action
Positive	Yes	Large	Deploy immediately
Positive	Yes	Small	Deploy, monitor long-term
Positive	No	Large	Consider running longer
Positive	No	Small	Dont deploy; iterate
Negative	Yes	Any	Revert; investigate root cause
Negative	No	Large	Consider running longer
Negative	No	Small	Dont deploy; move on
Flat	No	Tiny	Stop; focus elsewhere

Table 1: Decision framework for experiment results — the intersection of significance, effect size, and business context.

8.9 Debugging Checklist

When an experiment produces surprising results, run through this checklist:

- [01] Is randomization working correctly? Check covariate balance.
- [02] Is the metric computed consistently between groups?
- [03] Are there any instrumentation errors (missing data, wrong event firing)?
- [04] Is the sample size adequate for the observed effect?
- [05] Have there been any external events (campaigns, holidays, outages)?
- [06] Are there segments where the effect reverses (Simpson's Paradox)?
- [07] Is the test duration sufficient (at least one full business cycle)?
- [08] Are there any novelty or primacy effects in play?
- [09] Is the result practically significant, not just statistically significant?
- [10] Would you bet your bonus on this result?

Key Takeaway: Advanced A/B testing is not about using complex statistics — it's about avoiding common mistakes. The best experimenters spend more time on data quality and experimental design than on fancy analysis methods.

This guide has covered A/B testing from first principles through advanced implementation. Remember: experimentation is a mindset, not just a method. Every decision is a hypothesis. Test. Learn. Iterate. Repeat.